KI Dokumenten-Assistent

Async RAG Pipeline — Technische Dokumentation

FastAPI | ARQ / Redis | Qdrant | Claude Sonnet | Clean Architecture

Lokales, asynchrones Retrieval-Augmented Generation System für die Verarbeitung und semantische Befragung von PDF-Dokumenten.

1 Projektübersicht

Dieses System ermöglicht Nutzern, PDF-Dokumente hochzuladen und anschließend in natürlicher Sprache Fragen zu diesen Dokumenten zu stellen. Die Kernidee liegt in der entkoppelten, asynchronen Architektur: CPU-intensive Aufgaben (PDF-Parsing, Embedding-Generierung) blockieren nie die Haupt-API.

Eigenschaft	Bedeutung
Skalierbarkeit	API und Worker können unabhängig skaliert werden
Ausfallsicherheit	Fehlgeschlagene Jobs werden automatisch wiederholt
Performance	API antwortet sofort mit 202 Accepted – kein Blockieren
Wartbarkeit	Klare Trennung der Verantwortlichkeiten (Clean Architecture)

2 Architektur & Tech-Stack

Schichtenübersicht

Schicht	Technologie	Aufgabe
Frontend	Streamlit	Benutzeroberfläche für Upload & Chat
API	FastAPI + Pydantic	HTTP-Routing, Validierung, Orchestrierung
Worker	ARQ (Async Redis Queue)	PDF-Parsing, Chunking, Embedding-Generie
Message Broker	Redis	Job-Warteschlange zwischen API und Worke
Vektordatenbank	Qdrant (Docker)	Speicherung & Suche von Embeddings
Embeddings	fastembed · BAAI/bge-small	Lokale CPU-Embeddings, kostenlos

LLM	Anthropic Claude Sonnet	Textgenerierung via API

3 Systemvoraussetzungen

`Python 3.10+`	Laufzeitumgebung
`uv`	Moderner Python-Paketmanager (docs.astral.sh/uv)
`Docker & Docker Compose`	Für Redis und Qdrant
`Anthropic API-Key`	Für Claude (console.anthropic.com)

4 Installation & Setup

Schritt 1 — Repository klonen & Abhängigkeiten installieren

```
git clone <repo-url>
cd <projektverzeichnis>
uv sync
```

Hinweis: uv sync liest pyproject.toml und installiert alle Abhängigkeiten.

Schritt 2 — Infrastruktur starten

```
docker-compose up -d
```

Hinweis: Startet Redis und Qdrant als Docker-Container im Hintergrund.

Schritt 3 — Umgebungsvariablen konfigurieren

```
# Infrastruktur
REDIS_URL=redis://localhost:6379
QDRANT_HOST=localhost
QDRANT_PORT=6333

# KI Provider
ANTHROPIC_API_KEY=sk-ant-api03-...
```

Hinweis: Die .env-Datei niemals committen — sicherstellen, dass sie in .gitignore steht.

5 Das System starten

Aufgrund der entkoppelten Architektur müssen drei Komponenten in separaten Terminals gestartet werden.

Terminal 1 — API

```
uv run uvicorn src.interfaces.api.main:app --reload
```

FastAPI auf http://localhost:8000 · Swagger-Docs unter /docs

Terminal 2 — Background-Worker

```
uv run arq src.interfaces.worker.main.WorkerSettings
```

Beim ersten Start: Einmaliger Download des Embedding-Modells (~130 MB).

Terminal 3 — Frontend

```
uv run streamlit run src/interfaces/frontend/app.py
```

Streamlit öffnet sich automatisch unter http://localhost:8501

6 Bedienung & Datenfluss

Phase A — Dokument-Ingestion (Upload)

1	Nutzer lädt PDF über Streamlit hoch
2	Frontend → POST /documents/ → API speichert Datei, erstellt DB-Eintrag (PENDING), schickt job_id → Redis
3	API antwortet sofort mit HTTP 202 Accepted — Nutzer muss nicht warten
4	Worker liest Job → PyMuPDF (Parsing) → LangChain (Chunking) → fastembed (Embeddings)
5	Vektoren in Qdrant gespeichert → DB-Status: COMPLETED

Phase B — Query (Chat)

1	Nutzer stellt Frage im Streamlit-Chat
2	Frontend → POST /query/ → API wandelt Frage in Embedding-Vektor (lokal)
3	Qdrant: Top-3 semantisch ähnlichste Text-Chunks werden abgerufen
4	API baut Prompt (Kontext + Frage) → Claude Sonnet API
5	Claude generiert Antwort → Streamlit zeigt Antwort + Quellen an

7 Projektstruktur

Das Projekt folgt den Prinzipien der Clean Architecture, um Abhängigkeiten sauber zu trennen und einzelne Schichten unabhängig testbar und austauschbar zu halten.

```
projekt/
  src/
    interfaces/          # Kontaktpunkte nach außen
      api/               # FastAPI Router & Endpunkte
        main.py
      worker/            # ARQ Worker Settings & Tasks
        main.py
      frontend/          # Streamlit UI
        app.py
    infrastructure/      # Technische Querschnittsthemen
      config.py          # Pydantic Settings (.env)
      logging.py         # Strukturiertes Logging
      database/          # DB-Verbindungen & Modelle
  data/uploads/          # Temporärer Speicher für PDFs
  docker-compose.yml     # Redis & Qdrant
  pyproject.toml         # Abhängigkeiten (uv)
  .env                   # Lokale Konfiguration (nicht committen!)
  README.md              # Projektdokumentation
```

Architektur-Prinzipien

Dependency Inversion	Business-Logi VectorStore-In
Pydantic-Validierung	Strikte Datenv abgewiesen, d
Environment-Konfiguration	Alle Secrets ur